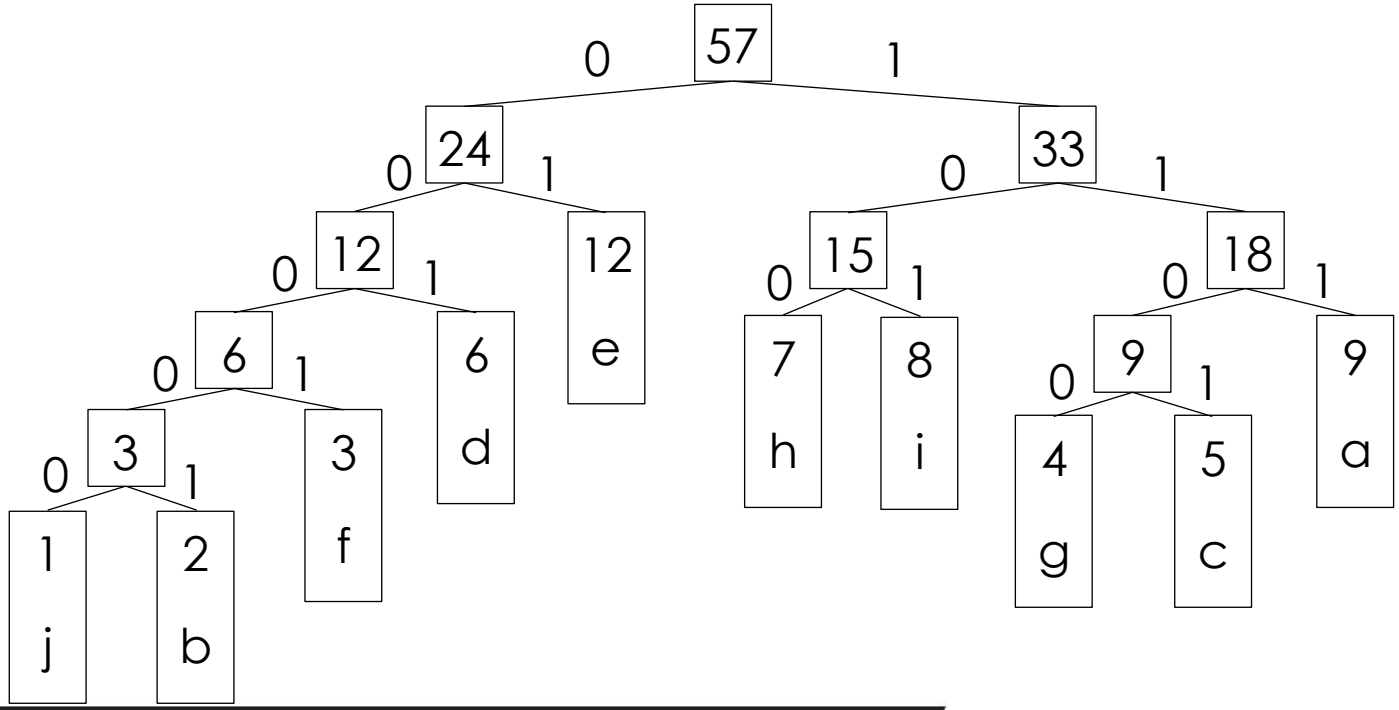
THE UNIVERSITY OF ALABAMA®

WHERE LEGENDS ARE MADE®

Huffman Codes

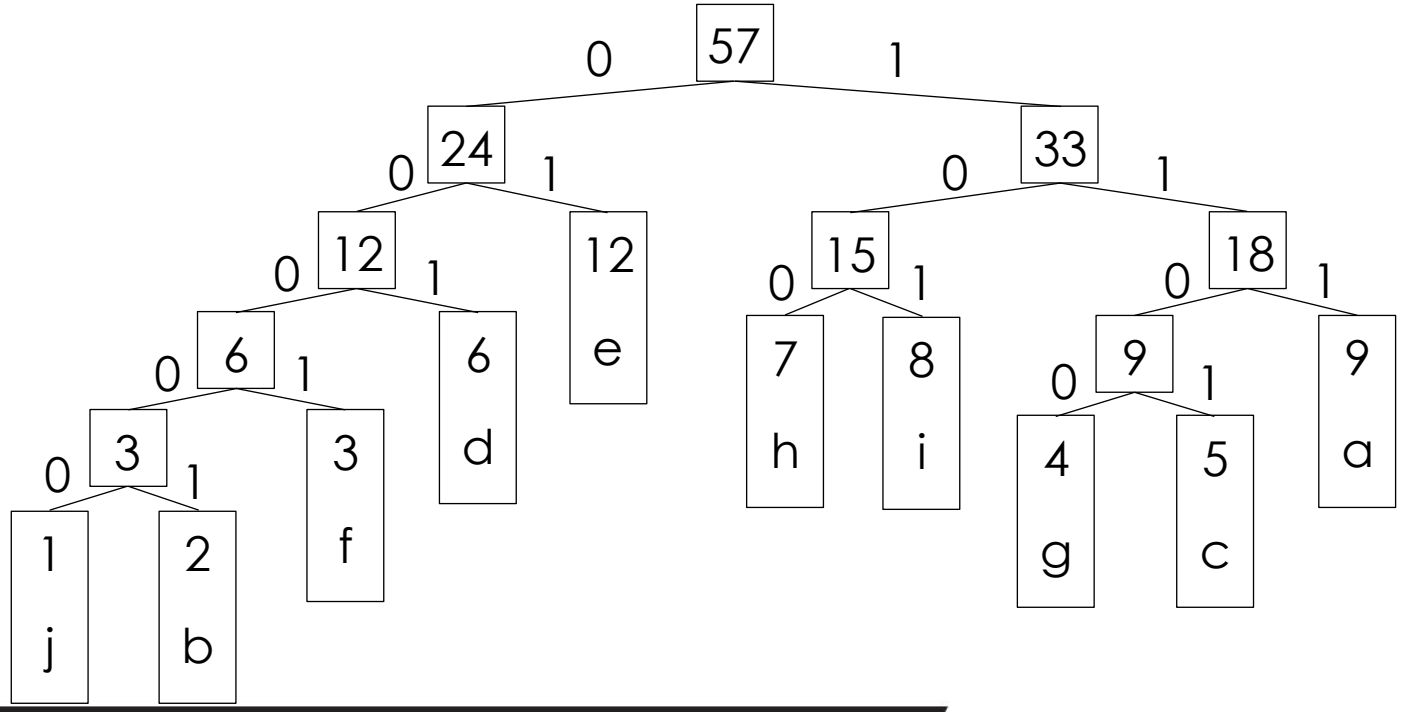
Greedy Huffman Algorithm

A	a	b	c	d	e	f	g	h	i	j
F	9	2	5	6	12	3	4	7	8	1
C	?									



Greedy Huffman Algorithm

A	a	b	c	d	e	f	g	h	i	j
F	9	2	5	6	12	3	4	7	8	1
C	111	00001	1101	001	01	0001	1100	100	101	00000



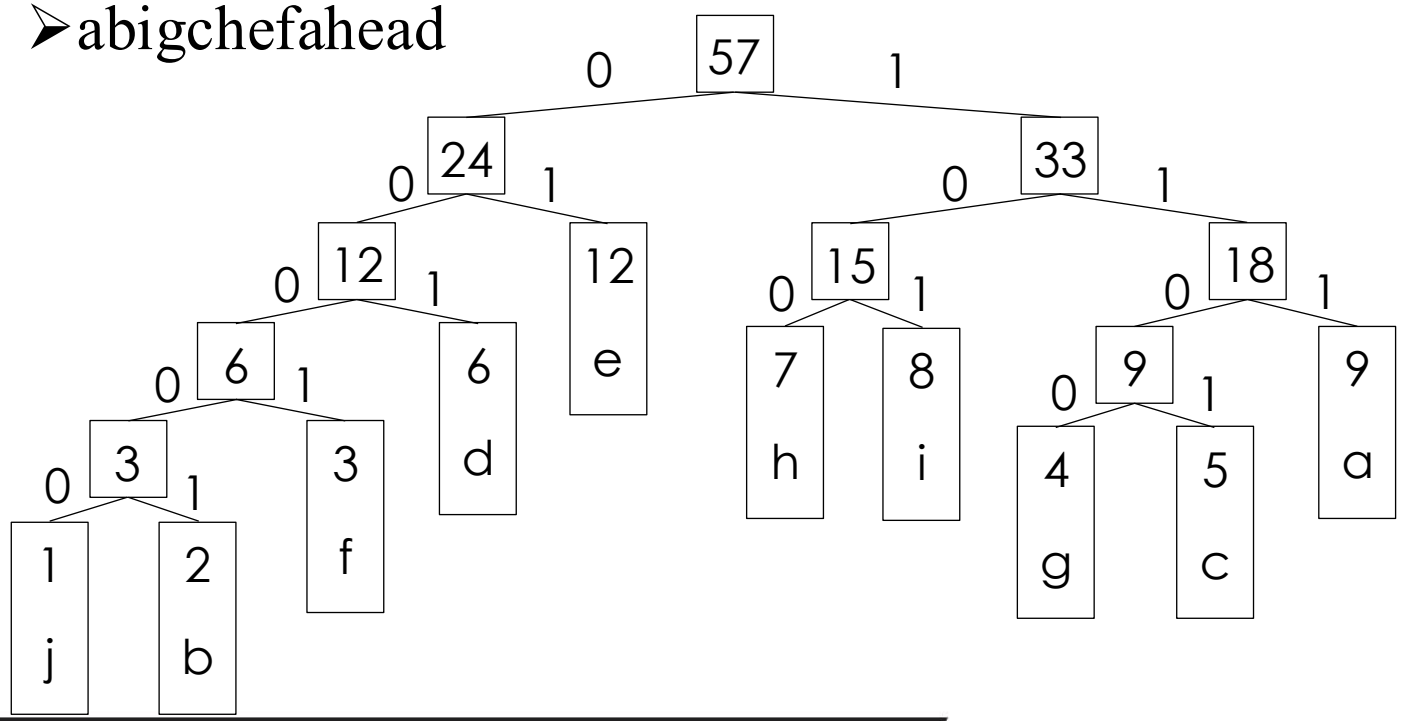
Encode

A	a	b	c	d	e	f	g	h	i	j
F	9	2	5	6	12	3	4	7	8	1
C	111	00001	1101	001	01	0001	1100	100	101	00000

- To encode a message, replace each character A[k] by its corresponding binary code C[k]
- Example
  - abigchefahead
  - 111000011011100110110001000111110001111001

Decode

- Example
  - 111000011011100110110001000111110001111001
  - abigchefahead



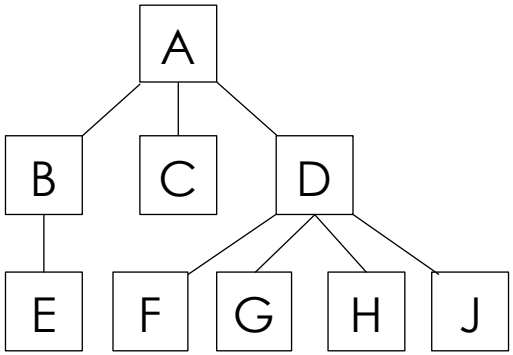
Non-Binary Trees & Traversals

Traversals

- Pre-order
- Post-order
- Level-order

Preorder Traversal

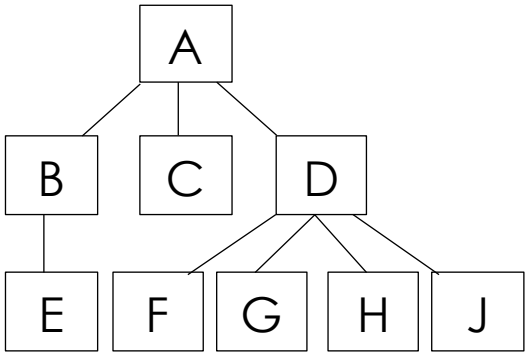
```
void preorder(Node p) {
  if (p == null) return;
  visit(p);
  preorder(p.leftmostChild);
  preorder(p.rightSibling);
}
```



Prefix: A <u>B E</u> <u>C</u> <u>D F G H J</u>

Postorder Traversal

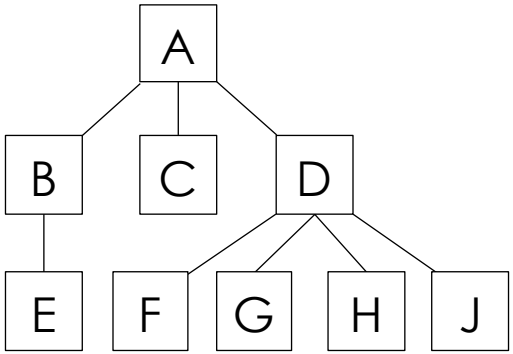
```
void postorder(Node p) {
  if (p == null) return;
  postorder(p.leftmostChild);
  visit(p);
  postorder(p.rightSibling);
}
```



Postfix: E B C F G H J D A

Level-order Traversal

```
void levelOrder(Tree T) {
  Queue Q();
  Q.enqueue(T.root);
  while (not Q.isEmpty()) {
    Node p = Q.dequeue();
    visit(p);
    for (Node c = p.leftmostChild; c != null; c = c.rightSibling)
      Q.enqueue(c);
  }
}
```



Level-order: A <u>B C D</u> <u>E F G H J</u>

Red-Black Trees (Insertion)

Properties of red-black trees

A red-black tree is a binary search tree that satisfies the following **Red-black** properties:

1. Every node is either red or black.
2. The root is black.
3. Every leaf (NIL) is black.
4. If a node is red, then both its children are black.
5. For each node, all simple paths from the node to descendant leaves contain the same number of black nodes

Proposition: The height of a red-black tree with n internal nodes is $\leq 2 * \lg(n+1) = O(\lg n)$

Rotations

```
LEFT-ROTATE(T, x)
y = x.right                    // set y
x.right = y.left               // turn y's left subtree into x's right subtree
if y.left ≠ T.nil
    y.left.p = x
y.p = x.p                      // link x's parent to y
if x.p == T.nil
    T.root = y
elseif x == x.p.left
    x.p.left = y
else x.p.right = y
y.left = x                     // put x on y's left
x.p = y
```

Insertion

```
RB-INSERT(T, z)
y = T.nil
x = T.root
while x ≠ T.nil
    y = x
    if z.key < x.key
        x = x.left
    else x = x.right
z.p = y
if y == T.nil
    T.root = z
elseif z.key < y.key
    y.left = z
else y.right = z
z.left = T.nil
z.right = T.nil
z.color = RED
RB-INSERT-FIXUP(T, z)
```

Insertion

The **RB-INSERT-FIXUP** algorithm ensures that the tree remains balanced after the insertion of a new node z. The key properties of a Red-Black Tree are:

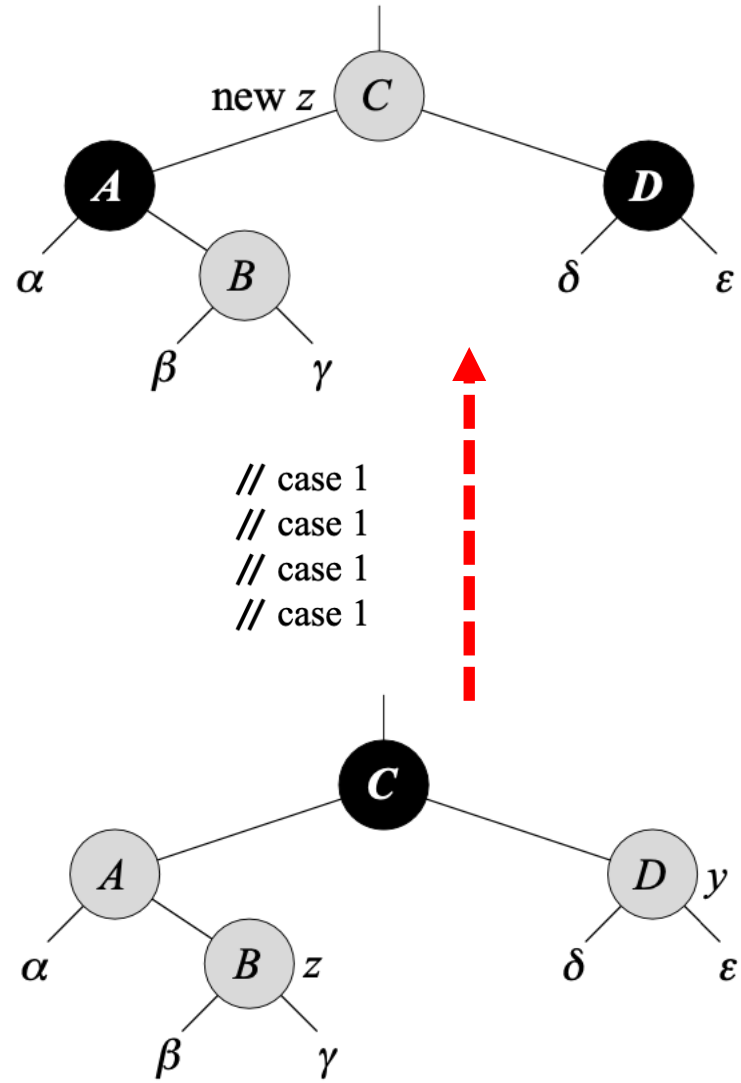
1. Every node is either **red** or **black**.
2. The root is always **black**.
3. Every leaf (NIL node) is **black**.
4. If a node is **red**, both of its children must be **black**.
5. For each node, all paths from the node to its descendant leaves must have the same number of black nodes (called the black-height).

After inserting a node, some of these properties might be violated. The algorithm works to restore the properties by adjusting the tree structure through **color changes** and **rotations**.

```
RB-INSERT-FIXUP(T, z)
while z.p.color == RED
    if z.p == z.p.p.left
        y = z.p.p.right
        if y.color == RED
            z.p.color = BLACK                    // case 1
            y.color = BLACK                      // case 1
            z.p.p.color = RED                    // case 1
            z = z.p.p                            // case 1
        else if z == z.p.right
                z = z.p                          // case 2
                LEFT-ROTATE(T, z)                // case 2
            z.p.color = BLACK                    // case 3
            z.p.p.color = RED                    // case 3
            RIGHT-ROTATE(T, z.p.p)               // case 3
    else (same as then clause
            with "right" and "left" exchanged)
T.root.color = BLACK
```

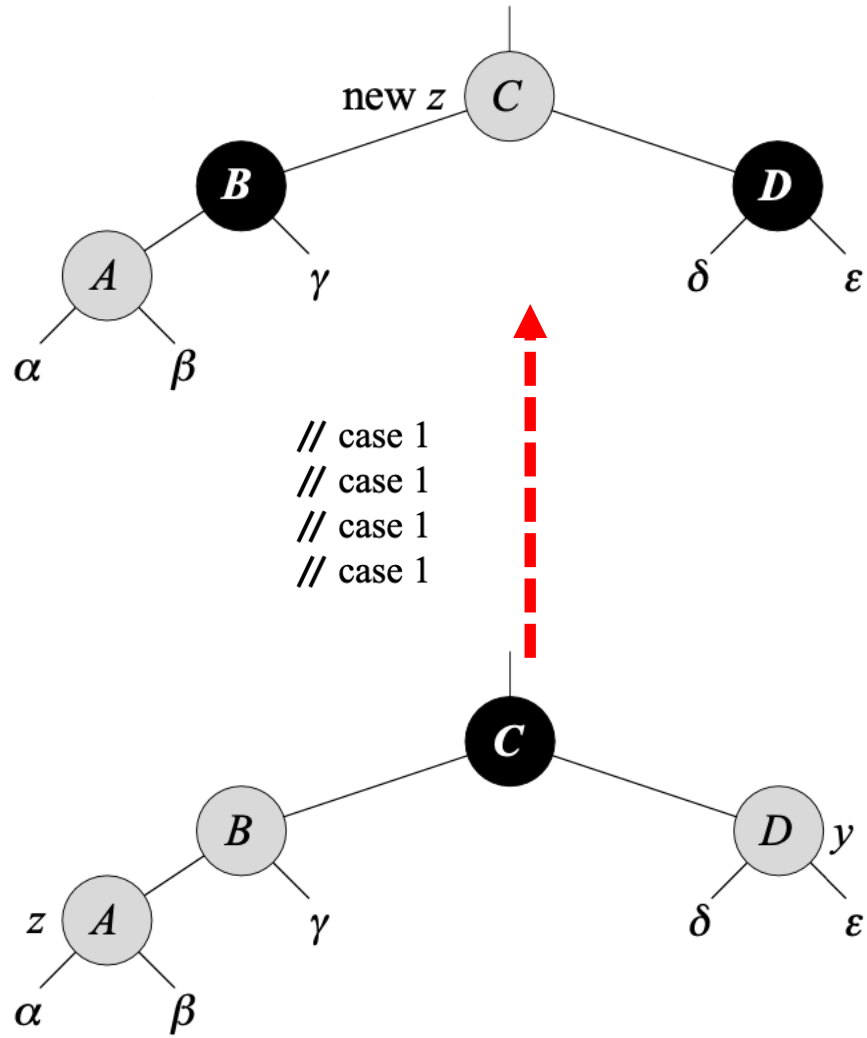
Insertion

```
RB-INSERT-FIXUP(T, z)
while z.p.color == RED
    if z.p == z.p.p.left
        y = z.p.p.right
        if y.color == RED
            z.p.color = BLACK
            y.color = BLACK
            z.p.p.color = RED
            z = z.p.p
        else if z == z.p.right
                z = z.p
                LEFT-ROTATE(T, z)
            z.p.color = BLACK
            z.p.p.color = RED
            RIGHT-ROTATE(T, z.p.p)
    else (same as then clause
            with "right" and "left" exchanged)
T.root.color = BLACK
```

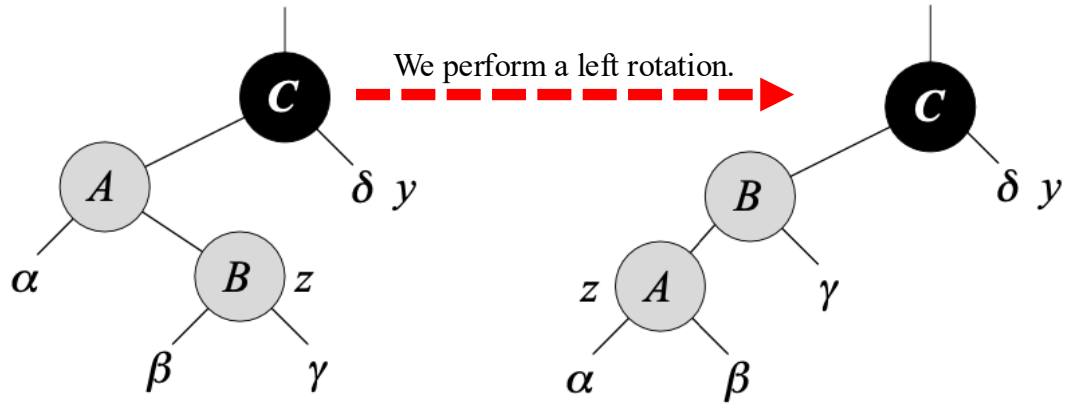


Insertion

```
RB-INSERT-FIXUP(T, z)
while z.p.color == RED
    if z.p == z.p.p.left
        y = z.p.p.right
        if y.color == RED
            z.p.color = BLACK
            y.color = BLACK
            z.p.p.color = RED
            z = z.p.p
        else if z == z.p.right
                z = z.p
                LEFT-ROTATE(T, z)
            z.p.color = BLACK
            z.p.p.color = RED
            RIGHT-ROTATE(T, z.p.p)
    else (same as then clause
            with "right" and "left" exchanged)
T.root.color = BLACK
```

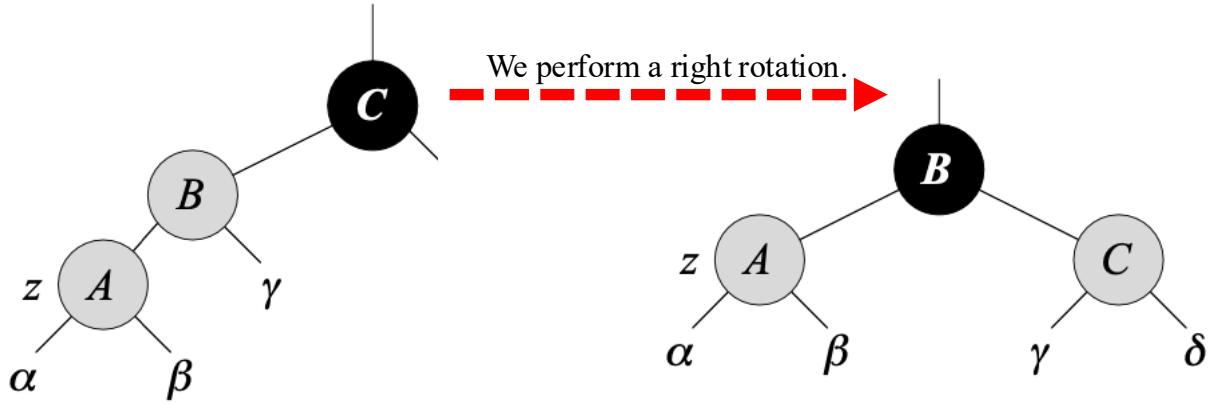


Insertion




```
RB-INSERT-FIXUP(T, z)
while z.p.color == RED
    if z.p == z.p.p.left
        y = z.p.p.right
        if y.color == RED
            z.p.color = BLACK
            y.color = BLACK
            z.p.p.color = RED
            z = z.p.p
        else if z == z.p.right
                z = z.p                                  // case 2
                LEFT-ROTATE(T, z)                        // case 2
            z.p.color = BLACK                            // case 3
            z.p.p.color = RED                            // case 3
            RIGHT-ROTATE(T, z.p.p)                       // case 3
    else (same as then clause
            with "right" and "left" exchanged)
T.root.color = BLACK
```

Insertion




```
RB-INSERT-FIXUP(T, z)
while z.p.color == RED
    if z.p == z.p.p.left
        y = z.p.p.right
        if y.color == RED
            z.p.color = BLACK
            y.color = BLACK
            z.p.p.color = RED
            z = z.p.p
        else if z == z.p.right
                z = z.p                                  // case 2
                LEFT-ROTATE(T, z)                        // case 2
            z.p.color = BLACK                            // case 3
            z.p.p.color = RED                            // case 3
            RIGHT-ROTATE(T, z.p.p)                       // case 3
    else (same as then clause
            with "right" and "left" exchanged)
T.root.color = BLACK
```

Red-Black Trees (Deletion)

Red-Black Trees - Deletion

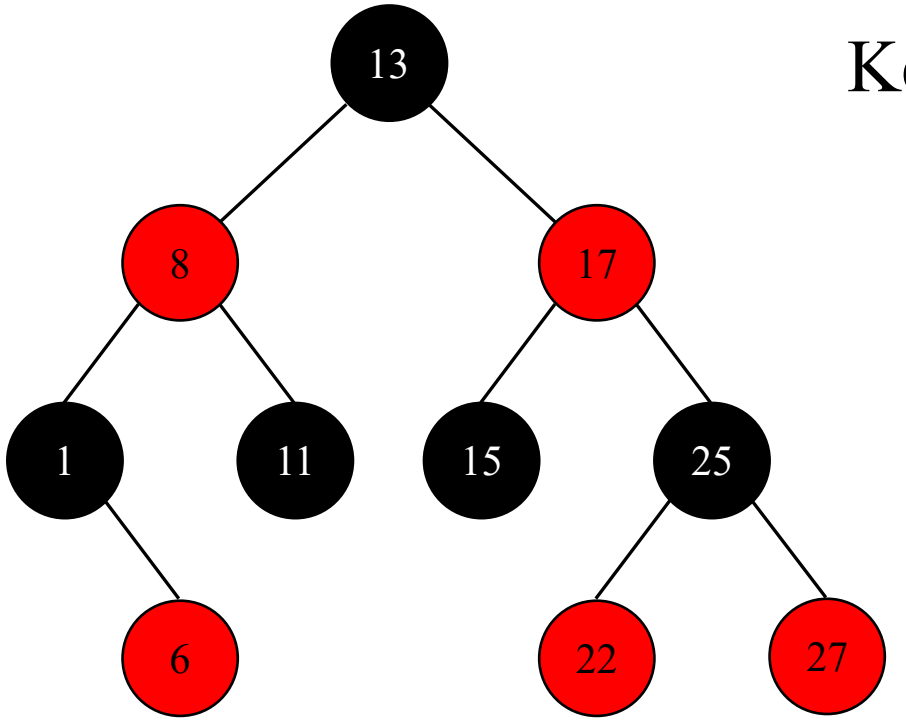
```
RB-DELETE(T, z)
y = z
y-original-color = y.color
if z.left == T.nil
    x = z.right
    RB-TRANSPLANT(T, z, z.right)
elseif z.right == T.nil
    x = z.left
    RB-TRANSPLANT(T, z, z.left)
else y = TREE-MINIMUM(z.right)
    y-original-color = y.color
    x = y.right
    if y.p == z
        x.p = y
    else RB-TRANSPLANT(T, y, y.right)
        y.right = z.right
        y.right.p = y
    RB-TRANSPLANT(T, z, y)
    y.left = z.left
    y.left.p = y
    y.color = z.color
if y-original-color == BLACK
    RB-DELETE-FIXUP(T, x)
```

```
RB-TRANSPLANT(T, u, v)
if u.p == T.nil
    T.root = v
elseif u == u.p.left
    u.p.left = v
else u.p.right = v
v.p = u.p
```

```
TREE-MINIMUM(x)
while x.left ≠ NIL
    x = x.left
return x
```

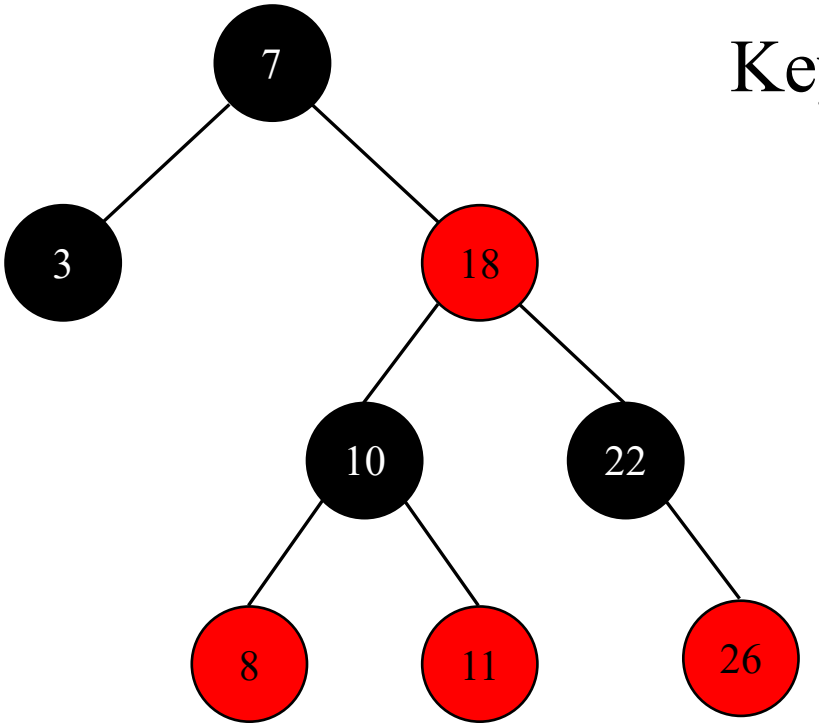
```
RB-DELETE-FIXUP(T, x)
while x ≠ T.root and x.color == BLACK
    if x == x.p.left
        w = x.p.right
        if w.color == RED
            w.color = BLACK                                   // case 1
            x.p.color = RED                                   // case 1
            LEFT-ROTATE(T, x.p)                               // case 1
            w = x.p.right                                     // case 1
        if w.left.color == BLACK and w.right.color == BLACK
            w.color = RED                                     // case 2
            x = x.p                                           // case 2
        else if w.right.color == BLACK
                w.left.color = BLACK                          // case 3
                w.color = RED                                 // case 3
                RIGHT-ROTATE(T, w)                            // case 3
                w = x.p.right                                 // case 3
            w.color = x.p.color                               // case 4
            x.p.color = BLACK                                 // case 4
            w.right.color = BLACK                             // case 4
            LEFT-ROTATE(T, x.p)                               // case 4
            x = T.root                                        // case 4
    else (same as then clause with "right" and "left" exchanged)
x.color = BLACK
```

Deletion



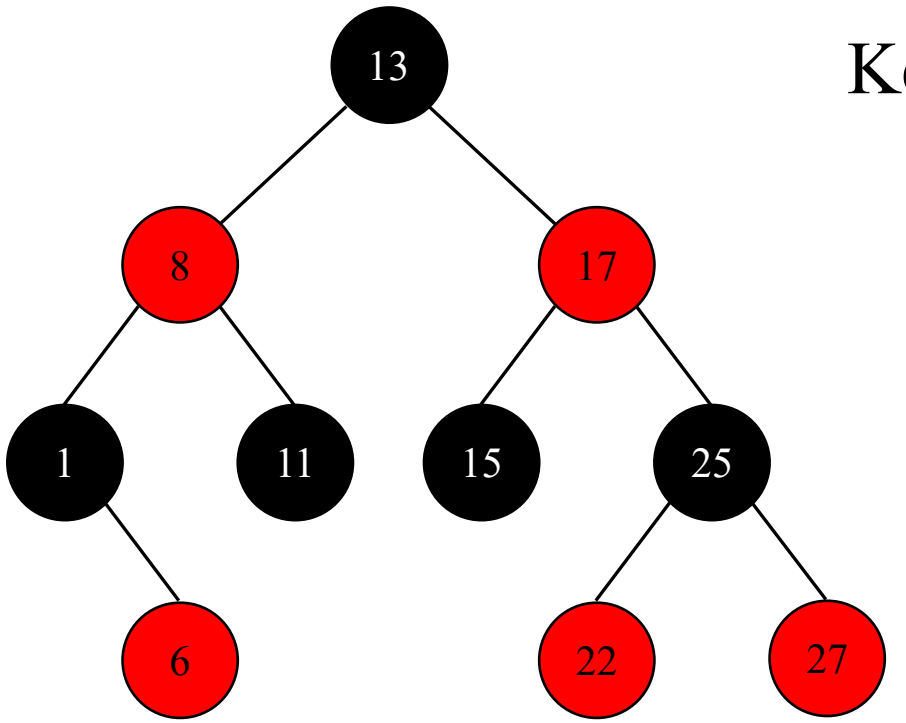
Key: 8

Deletion



Key: 3

Deletion



Key: 11

B-Trees
(Insertion & Search)

A simple B-tree

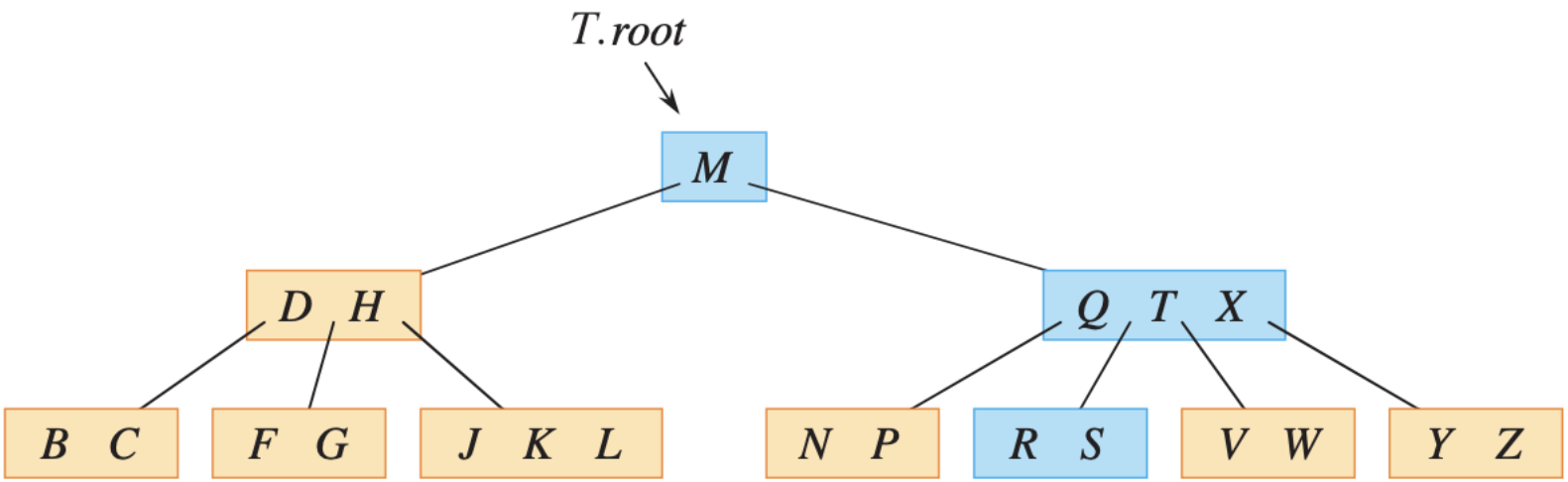


Figure: A B-tree whose keys are the consonants of English.

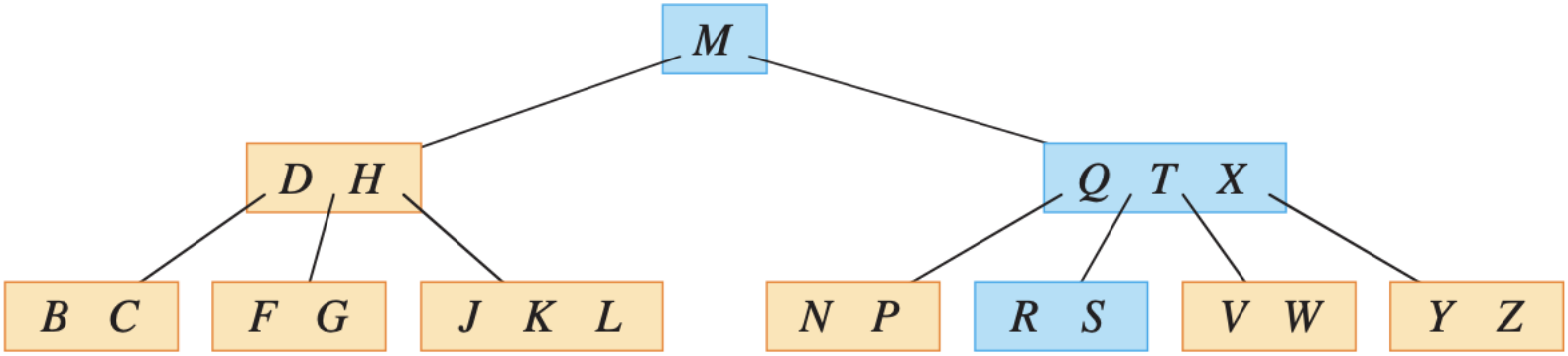
Five Structural Properties

1. Every node x has the following attributes:

a. $x.n$, the number of keys currently stored in node x,

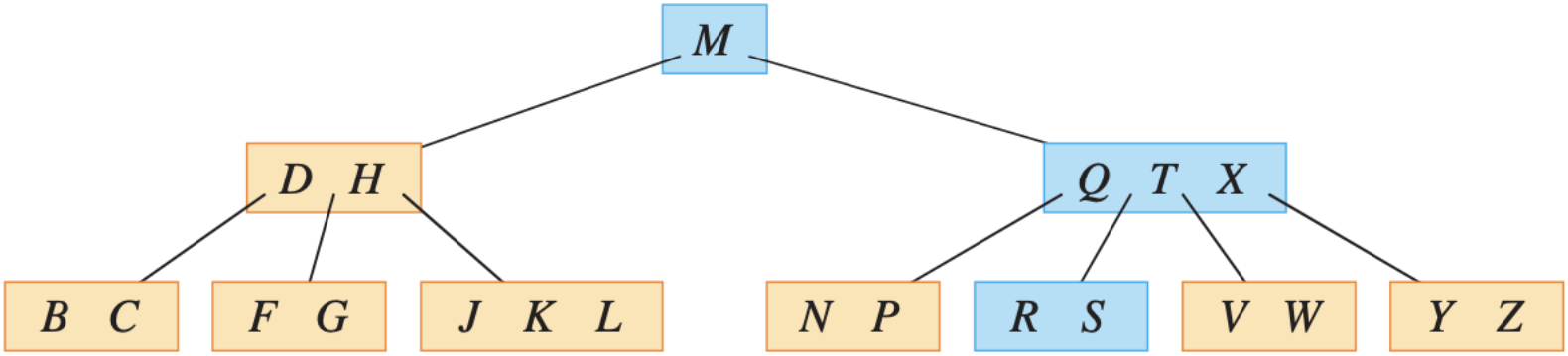
b. the $x.n$ keys themselves, $x.key_1, x.key_2, \ldots, x.key_{x.n}$, stored in monotonically increasing order, so that $x.key_1 \leq x.key_2 \leq \ldots \leq x.key_{x.n}$,

c. $x.leaf$, a boolean value that is TRUE if x is a leaf and FALSE if x is an internal node.



Five Structural Properties

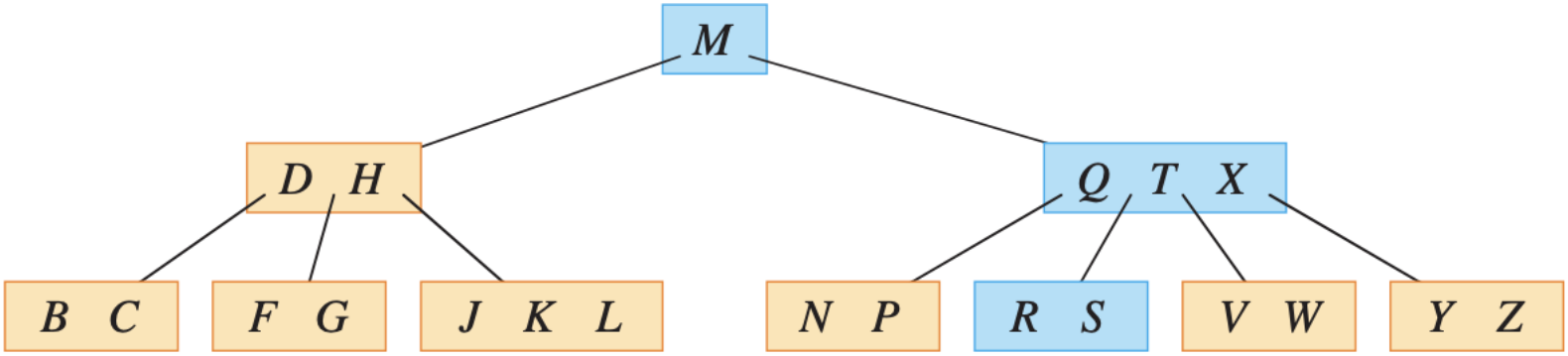
2. Each internal node x also contains $x.n + 1$ pointers $x.c_1, x.c_2, \ldots, x.c_{x.n+1}$ to its children. Leaf nodes have no children, and so their c_i attributes are undefined.



Five Structural Properties

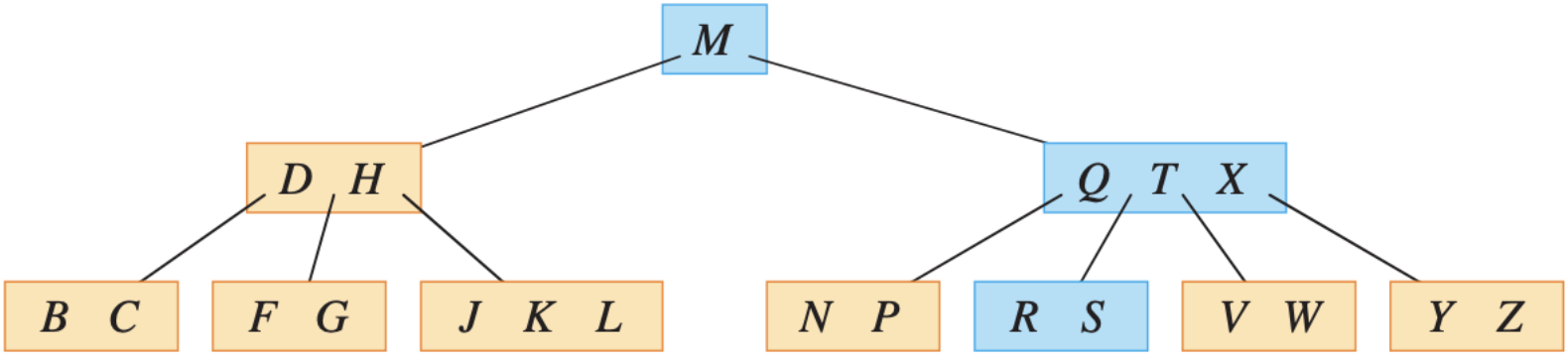
3. The keys $x.\text{key}_i$ separate the ranges of keys stored in each subtree: if k_i is any key stored in the subtree with root $x.c_i$, then

$$k_1 \leq x.\text{key}_1 \leq k_2 \leq x.\text{key}_2 \leq \ldots \leq x.\text{key}_{x.n} \leq k_{x.n+1}$$



Five Structural Properties

4. All leaves have the same depth, which is the tree's height h.



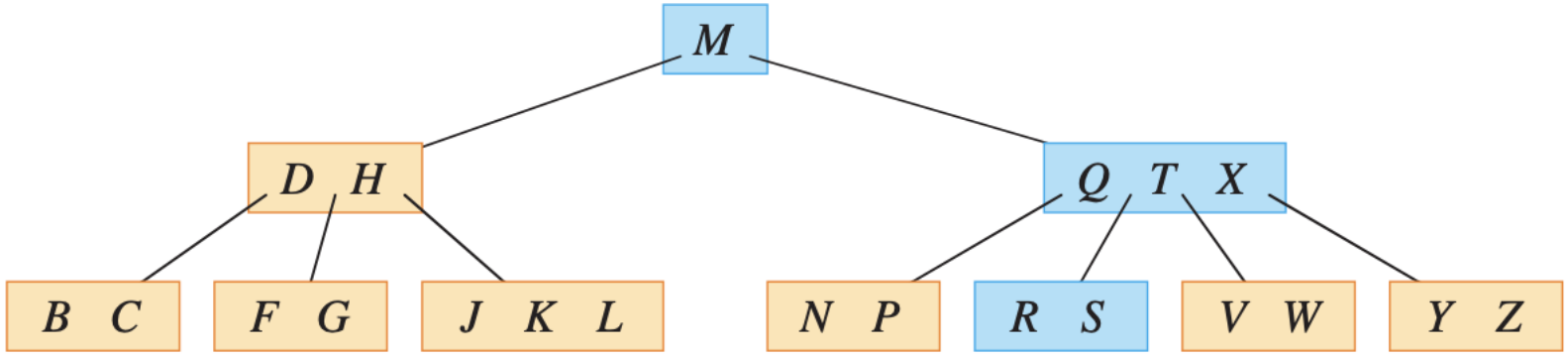
Five Structural Properties

5. Nodes have lower and upper bounds on the number of keys they can contain, expressed in terms of a fixed integer $t \geq 2$ called the ***minimum degree*** of the B-tree:

   a. Every node other than the root must have at least $t - 1$ keys. Every internal node other than the root thus has at least t children. If the tree is nonempty, the root must have at least one key.

   b. Every node may contain at most $2t - 1$ keys. Therefore, an internal node may have at most $2t$ children. We say that a node is ***full*** if it contains exactly $2t - 1$ keys.[3]

The simplest B-tree occurs when $t = 2$. Every internal node then has either 2, 3, or 4 children, and it is a ***2-3-4 tree***. In practice, however, much larger values of t yield B-trees with smaller height.



Search

```
B-TREE-SEARCH(x, k)
    i = 1
    while i <= x.n and k > x.key[i]
        i = i + 1
    if i <= x.n and k == x.key[i]
        return (x, i)
    elseif x.leaf
        return NIL
    else DISK-READ(x.c[i])
    return B-TREE-SEARCH(x.c[i], k)
```

Inserting Operation

Insertion Operation

```
B-TREE-CREATE(T)
    x = ALLOCATE-NODE()
    x.leaf = TRUE
    x.n = 0
    DISK-WRITE(x)
    T.root = x
```

Insertion Operation

```
B-TREE-INSERT(T, k)
    r = T.root
    if r.n == 2t - 1
        s = B-TREE-SPLIT-ROOT(T)
        B-TREE-INSERT-NONFULL(s, k)
    else B-TREE-INSERT-NONFULL(r, k)
```

Insertion Operation

```
B-TREE-SPLIT-ROOT(T)
s = ALLOCATE-NODE()
s.leaf = FALSE
s.n = 0
s.c1 = T.root
T.root = s
B-TREE-SPLIT-CHILD(s, 1)
return s
```

Insertion Operation

```
B-TREE-SPLIT-CHILD(x, i)
y = x.c_i
z = ALLOCATE-NODE()
z.leaf = y.leaf
z.n = t - 1
for j = 1 to t - 1
  z.key_j = y.key_(j+t)
if not y.leaf
  for j = 1 to t
    z.c_j = y.c_(j+t)
y.n = t - 1
for j = x.n + 1 downto i + 1
  x.c_(j+1) = x.c_j
x.c_(i+1) = z
for j = x.n downto i
  x.key_(j+1) = x.key_j
x.key_i = y.key_t
x.n = x.n + 1
DISK-WRITE(y)
DISK-WRITE(z)
DISK-WRITE(x)
```

Insertion Operation

```
B-TREE-INSERT-NONFULL(x, k)
i = x.n
if x.leaf
  while i >= 1 and k < x.key_i
    x.key_(i+1) = x.key_i
    i = i - 1
  x.key_(i+1) = k
  x.n = x.n + 1
  DISK-WRITE(x)
else
  while i >= 1 and k < x.key_i
    i = i - 1
  i = i + 1
  DISK-READ(x.c_i)
  if x.c_i.n == 2t - 1
    B-TREE-SPLIT-CHILD(x, i)
    if k > x.key_i
      i = i + 1
  B-TREE-INSERT-NONFULL(x.c_i, k)
```

B-Trees (Deletion)

Deletion

Case 1: ***The search arrives at a leaf node*** x. If x contains key k, then delete k from x. If x does not contain key k, then k was not in the B-tree and nothing else needs to be done.

Case 2: ***The search arrives at an internal node*** x *that contains key* k. Let $k = x.\mathrm{key}_i$. One of the following three cases applies, depending on the number of keys in $x.c_i$ (the child of x that precedes k) and $x.c_{i+1}$ (the child of x that follows k).

Case 3: ***The search arrives at an internal node*** x *that does not contain key* k. Continue searching down the tree while ensuring that each node visited has at least t keys. To do so, determine the root $x.c_i$ of the appropriate subtree that must contain k, if k is in the tree at all. If $x.c_i$ has only $t-1$ keys, execute case 3a or 3b as necessary to guarantee descending to a node containing at least t keys. Then finish by recursing on the appropriate child of x.

Case 2

Case 2: ***The search arrives at an internal node*** x *that contains key* k. Let $k = x.\text{key}_i$. One of the following three cases applies, depending on the number of keys in $x.c_i$ (the child of x that precedes k) and $x.c_{i+1}$ (the child of x that follows k).

Case 2a: *$x.c_i$ has at least t keys.* Find the predecessor k' of k in the subtree rooted at $x.c_i$. Recursively delete k' from $x.c_i$, and replace k by k' in x. (Key k' can be found and deleted in a single downward pass.)

Case 2b: *$x.c_i$ has $t-1$ keys and $x.c_{i+1}$ has at least t keys.* This case is symmetric to case 2a. Find the successor k' of k in the subtree rooted at $x.c_{i+1}$. Recursively delete k' from $x.c_{i+1}$, and replace k by k' in x. (Again, finding and deleting k' can be done in a single downward pass.)

Case 2c: *Both $x.c_i$ and $x.c_{i+1}$ have $t-1$ keys.* Merge k and all of $x.c_{i+1}$ into $x.c_i$, so that x loses both k and the pointer to $x.c_{i+1}$, and $x.c_i$ now contains $2t-1$ keys. Then free $x.c_{i+1}$ and recursively delete k from $x.c_i$.

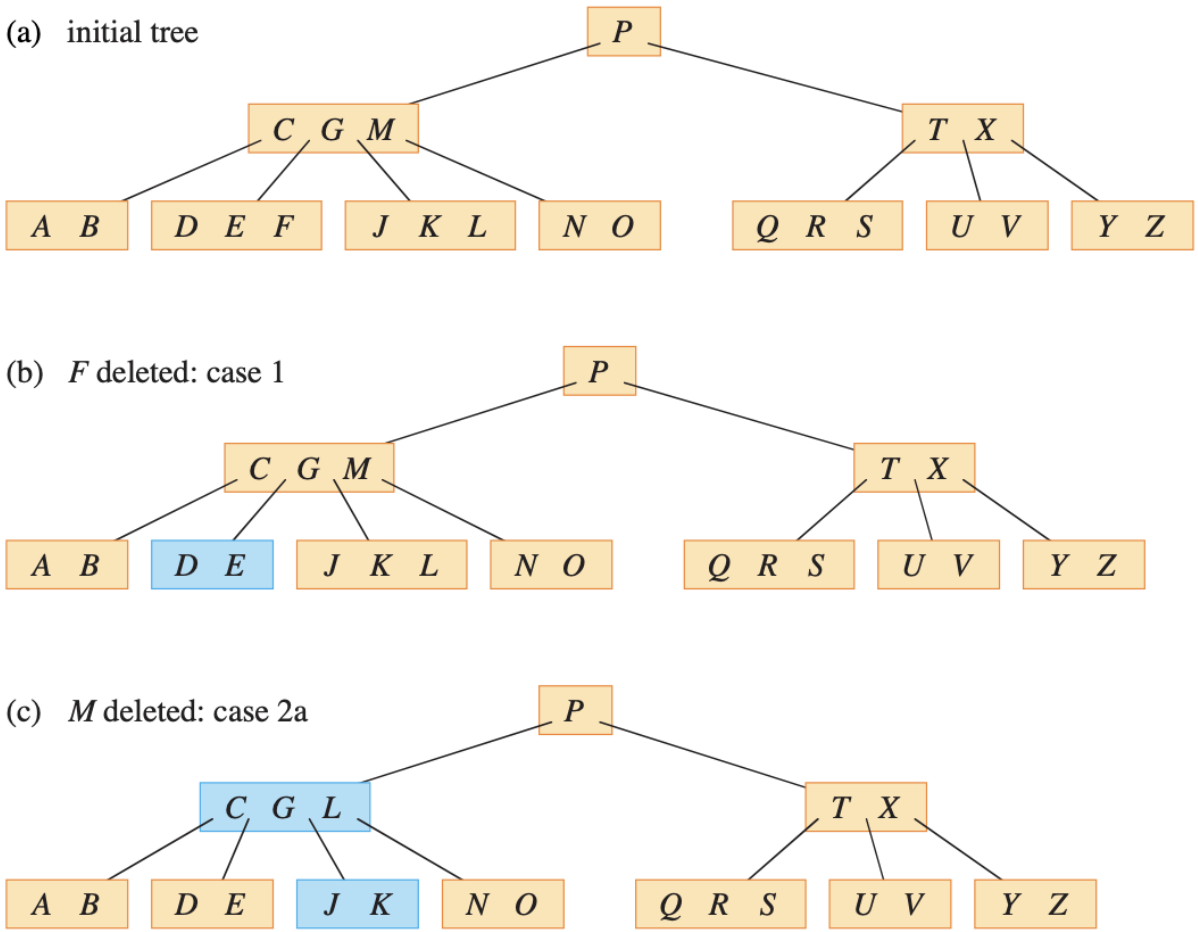
Case 3

Case 3: ***The search arrives at an internal node*** x *that does not contain key* k. Continue searching down the tree while ensuring that each node visited has at least t keys. To do so, determine the root $x.c_i$ of the appropriate subtree that must contain k, if k is in the tree at all. If $x.c_i$ has only $t-1$ keys, execute case 3a or 3b as necessary to guarantee descending to a node containing at least t keys. Then finish by recursing on the appropriate child of x.

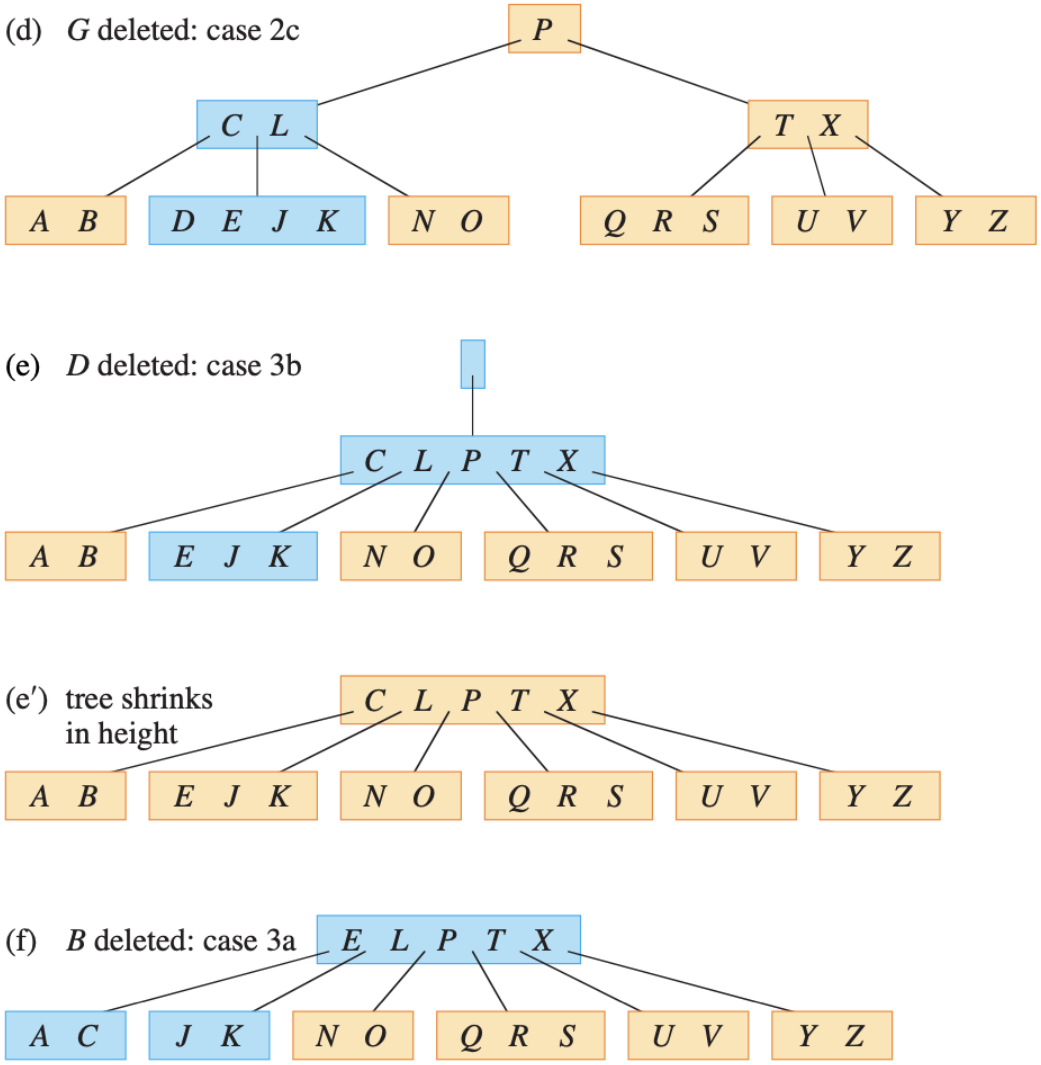
Case 3a: *$x.c_i$ has only $t-1$ keys but has an immediate sibling with at least t keys.* Give $x.c_i$ an extra key by moving a key from x down into $x.c_i$, moving a key from $x.c_i$'s immediate left or right sibling up into x, and moving the appropriate child pointer from the sibling into $x.c_i$.

Case 3b: *$x.c_i$ and each of $x.c_i$'s immediate siblings have $t-1$ keys.* (It is possible for $x.c_i$ to have either one or two siblings.) Merge $x.c_i$ with one sibling, which involves moving a key from x down into the new merged node to become the median key for that node.

Deletion - Practices



Deletion - Practices



Questions?